Script

Slide 3: Project Overview & Introduction

Talking Points:

- **The Hook:** Traditional messaging apps like WhatsApp or Slack are fantastic, but they are often bloated, resource-intensive, and prone to distracting notifications.
- **The Solution:** We built a real-time, asynchronous Terminal-Based Chat Application. It delivers the instant responsiveness of modern GUI platforms, but operates entirely within a minimal-resource command-line environment.
- **Target Audience:** This is specifically engineered for developers, system administrators, and CLI enthusiasts who want to maintain an uninterrupted workflow without leaving their terminal.

Core objectives: Talking Points:

Our engineering goals were centered around speed, security, and native feel.

- **Authentication:** We eliminated passwords entirely. Instead, we use a secure, session-based login via mobile numbers and transient One-Time Passwords (OTPs).
- **Real-Time Exchange:** We implemented full-duplex WebSockets to guarantee instant message delivery.
- **Persistence:** We ensured no data is lost by mapping all user and chat history to a persistent PostgreSQL relational database.
- **UI Experience:** We designed a reactive Terminal User Interface (TUI) that feels modern and responsive, never blocking user input.

Slide 4: The Technology Stack (Deep Dive)

Talking Points:

(Introduction) This is where the application really shines structurally. To make a terminal app feel as fast as a modern GUI app, we chose a highly modern, fully asynchronous Python stack across four key areas: the Backend, Frontend, Database, and Networking.

- **Backend (FastAPI & Uvicorn):** We utilized FastAPI running over an ASGI (Asynchronous Server Gateway Interface) server, which is Uvicorn.

  > **Technical Detail:** FastAPI acts as the "brain" routing our logic, while Uvicorn is the actual ASGI web server. Uvicorn runs the entire application on Python's `asyncio` event loop. Unlike older synchronous frameworks that block threads while waiting for

> a database query to finish, this FastAPI/Uvicorn combination uses Python's `async/await` mechanics. If the server needs to wait for a database write, Uvicorn instantly frees up that thread to handle another user's incoming message, giving us highly concurrent throughput.
>
> Uvicorn manages the raw HTTP requests; think of it like a waiter. It gets your request and goes to the chef. The chef here is FastAPI—Uvicorn gives the request to it, and then FastAPI gets exactly what is requested.

- **Frontend (Textual & Rich):** The UI is powered by Textual.

> **Technical Detail:** Traditional CLI scripts print lines sequentially. Textual, however, treats the terminal like a canvas using a reactive, event-driven engine. When a new message arrives, it calculates the difference and repaints only that specific bounding box, ensuring a completely flicker-free experience.
>
> **The Role of Rich:** While Textual handles the layout structure, Rich is the engine actually drawing the pixels. Rich is responsible for the refined typography, advanced color parsing, and the CSS-styled text formatting. Without Rich, a terminal is just monochrome text; with it, we achieve the polished, readable aesthetic of a modern graphical application right in the command line.

- **Database (PostgreSQL & SQLModel):** We use PostgreSQL as our permanent storage, driven by the `asyncpg` driver for non-blocking I/O.

> **Technical Detail:** We manage the schema with SQLModel, the library, which bridges SQLAlchemy (the ORM) (mapping Python classes to SQL tables) and Pydantic (handling strict data validation). This natively prevents SQL injection attacks and ensures malformed data never enters our system.

- **Networking (HTTPx & WebSockets):** This is how our data actually moves. We used a dual-protocol approach to maximize efficiency.
  - **The HTTP Layer (httpx):** The initial handshake and authentication (like requesting and verifying the OTP) are handled via standard, stateless HTTP requests using the `httpx` library. Because authentication is a one-time event per session, standard HTTP is perfectly efficient here.
  - **The WebSocket Layer:** Once a user is authenticated, we upgrade them to a persistent, full-duplex WebSocket connection.

> **Technical Detail:** Standard HTTP relies on "polling"—the client constantly asking the server, "Do I have new messages?" which wastes immense bandwidth and causes delays. WebSockets solve this by acting like a dedicated, open phone line. The connection stays open, allowing the server to push inbound messages directly to the user the exact millisecond they arrive, bypassing slow HTTP polling loops entirely.

Slide 5: System Architecture & Data Flow

Talking Points:

Our architecture splits traffic into two distinct channels: stateless HTTP and stateful WebSockets.

- **Phase 1: The HTTP Handshake.** The client first makes an asynchronous HTTP request to our backend (`/request-otp`). The server generates a code and caches it in memory. Once the user validates this code, the backend verifies their identity against the database.
- **Phase 2: The WebSocket Upgrade.** Upon successful login, the client drops the HTTP connection and opens a persistent WebSocket (`/ws/{phone_number}`).

  > **Technical Detail:** Standard HTTP requires "polling," which wastes bandwidth. WebSockets act like a dedicated phone line. The server keeps a live dictionary in memory (`active_connections`) mapping phone numbers directly to active socket objects.

- **Phase 3: Real-Time Routing.** When User A sends a message to User B, the server intercepts it, permanently logs it to PostgreSQL, checks the `active_connections` dictionary for User B's phone number, and instantly pushes the payload down their open pipe in a single asynchronous operation.

Slide 6: Database Schema (Entity Relationship)

Talking Points:

We kept the relational schema optimized and straightforward. It operates on a strictly 1-to-Many (1:N) relationship between Users and Messages.

- **Users Table:** Tracks the `id` (Primary Key), `phone_number` (Unique), and `is_verified` boolean.
- **Messages Table:** Contains its own `id` , the actual text content, a UTC timestamp, and two crucial Foreign Keys: `sender_id` and `receiver_id` .

This structure allows us to instantly access the chat logs between two clients. It uses an ORM, here SQLAlchemy, to get this data from the database by eliminating the need for manual SQL and associated risks (SQL injections).

While putting the message or user information in the DB, the ORM allows us to create structured Python objects rather than raw SQL. This is done by passing validated messages (validated using Pydantic) to the database session.

Slide 7: Backend Implementation

Talking Points:

(Introduction) Our backend is divided into four main operational layers. It is designed to perfectly balance temporary, fast actions (like logging in) with persistent, long-running actions (like streaming messages).

1. **REST Endpoints (The Stateless Gateway):** We use standard, stateless HTTP routes for the authentication handshakes and retrieving chat history.

   > **Technical Detail:** When a user hits the `/request-otp` endpoint, the server generates a token and drops it into a temporary, in-memory dictionary called `otp_storage`. We keep this in memory rather than the database because OTPs are highly transient. Once verified at the `/verify-otp` endpoint, the token is instantly wiped from RAM to maintain security and keep memory usage low.

2. **WebSocket Gateway (The Stateful Pipeline):** This is where the real-time magic happens. We maintain a persistent socket at the `/ws/{phone_number}` route.

   > **Technical Detail:** The backend maintains a critical in-memory map called `active_connections`. When a user successfully authenticates, their live WebSocket object is saved in this dictionary, keyed directly by their phone number. This means when a message arrives, the server doesn't have to query the database to find out where to send it; it just looks up the recipient's phone number in this dictionary for instantaneous, direct routing.

3. **Persistence Layer (Non-Blocking I/O):** Even though our messages stream over WebSockets, we still need to save them permanently. This is handled by SQLModel and PostgreSQL.

   > **Technical Detail:** To ensure our fast WebSocket streams aren't slowed down by database writes, we execute our SQLModel definitions strictly through the `asyncpg` driver. This driver guarantees that all reads and writes to PostgreSQL are completely non-blocking, that is, asynchronous.

4. **Concurrency Model (The Event Loop):**

   > **Technical Detail:** The entire backend server operates on Python's `asyncio` event loop. Because it is served by Uvicorn over the ASGI (Asynchronous Server Gateway Interface), a single server thread can handle thousands of concurrent WebSocket connections and REST requests simultaneously without locking up.

Slide 8: Source Modules & Code Architecture

Talking Points:

(Introduction) As the project grew in complexity, we strictly adhered to a core software engineering principle: Separation of Concerns. Instead of writing one massive, unreadable script, we divided the application into four distinct, highly specialized modules. Each file has exactly one job.

1. `main.py` **(The Brain & Traffic Controller)**
   - **What it is:** This is the central hub of our backend application. It hosts the FastAPI application and the ASGI server loop.
   - **Technical Detail:** This file handles all the network routing. It exposes the stateless HTTP endpoints (`/request-otp` and `/verify-otp`) and manages the stateful WebSocket pipeline. When a message arrives, `main.py` is responsible for catching it, calling the database to save it, and then instantly looking up the recipient's socket in memory to push the message forward.
   - **The Analogy:** Think of this as the air traffic controller. It doesn't store the luggage or fly the planes; it just directs the massive flow of data exactly where it needs to go, the millisecond it arrives.
2. `client.py` **(The Terminal Experience)**
   - **What it is:** This is the frontend application that runs locally on the user's machine.
   - **Technical Detail:** It utilizes the Textual framework to render the reactive, CSS-styled terminal layout. Behind the scenes, it manages asynchronous tasks: it uses `httpx` to send login payloads, and it keeps a background WebSocket listener open to catch inbound messages without freezing the user's screen or interrupting their typing.
   - **The Analogy:** This is the dashboard and steering wheel of the car. It is the only part of the system the user actually interacts with, translating their keystrokes into commands the backend can understand.
3. `models.py` **(The Blueprint & Rulebook)**
   - **What it is:** This file defines the exact shape and strict rules for our data.
   - **Technical Detail:** Here, we define our User and Message structures using SQLModel. We specify primary keys, unique constraints (like phone numbers), and the foreign key relationships that link a message to its sender and receiver. Crucially, because it works with Pydantic (our validator), this file acts as our security guard. It actively validates incoming data, rejecting any malformed or malicious inputs before they can enter the database.
   - **The Analogy:** If the database is a secure vault, `models.py` is both the blueprint for how the vault is organized and the security guard checking IDs at the door.
4. `database.py` **(The Storage Pipe)**
   - **What it is:** This is the infrastructure bridge that connects our Python code to the PostgreSQL database.

- **Technical Detail:** It begins by configuring the asynchronous `asyncpg` engine via a connection string, ensuring the system can handle concurrent user traffic. On initialization, a function checks and generates our DB tables if they are not there. It provides an asynchronous generator (`get_session` function) that generates database sessions for `main.py` whenever a read or write is needed.
- **The Analogy:** This is the plumbing system. It doesn't decide what water flows through it—that is the job of `main.py` and `models.py` —it simply provides the secure, leak-proof pipes that allow data to flow safely into persistent storage.

Slide 9: User Authentication & Security Flow

Talking Points:

(Introduction) For authentication, we decided to completely eliminate traditional passwords. Storing passwords requires complex hashing algorithms and exposes the system to data leaks. Instead, we implemented a password-less, two-step verification mechanism utilizing a transient One-Time Password (OTP) keyed directly to the user's mobile number.

- **Step 1: Requesting the OTP (The Trigger)**
  - **The Flow:** The user inputs their phone number into the Textual client, which sends a stateless HTTP request to our backend's `/request-otp` endpoint.
  - **Technical Detail:** The FastAPI server instantly generates a random 6-digit code. Crucially, we do not store this code in our permanent PostgreSQL database. Instead, it is saved into a temporary, in-memory Python dictionary (the cache). This ensures the token is highly ephemeral—meaning it exists only in the server's RAM and can never permanently clutter our database.
- **Step 2: Verifying the OTP (Lazy Registration)**
  - **The Flow:** The user types the received code into the interface, and the client sends it to the `/verify-otp` endpoint. The server compares this input against the in-memory cache.
  - **Technical Detail:** If the code matches, we do something called "lazy registration." The system queries the PostgreSQL database; if it doesn't recognize the phone number, it instantly creates a brand-new user profile right then and there, marking it as `is_verified = True`. Immediately after verification, the system destroys the OTP token from the memory cache so it can never be reused.
- **Step 3: Establishing the Session (The Upgrade)**
  - **The Flow:** With the identity validated, the backend returns a success message to the client.
  - **Technical Detail:** The Textual interface reads this success state, drops the login screen, and renders the main chat workspace. Only at this exact moment does the

client initiate the persistent WebSocket connection to begin real-time communication.

> **How to summarize it quickly during the presentation:**
>
> "By utilizing an in-memory OTP cache and lazy database registration, we created a highly secure, password-less login flow. The system verifies users quickly via RAM, safely registers them in PostgreSQL, and instantly destroys the temporary access tokens to prevent reuse, all before a WebSocket is ever opened."

Slide 10: Real-Time Communication

Talking Points:

(Introduction) This slide represents the core feature of our application. Delivering real-time text in a terminal without freezing the user's keyboard input is a complex concurrency challenge. We did this by combining full-duplex WebSockets on the backend with a reactive asynchronous loop on the frontend.

1. **The Full-Duplex Pipeline (No Polling):**
   - **The Concept:** Once authentication completes, the client establishes a persistent, full-duplex WebSocket channel.
   - **Technical Detail:** Traditional web apps often use "polling," where the client constantly asks the server for new data. This is slow and wasteful. A full-duplex WebSocket is like an open telephone line; neither side has to hang up. It allows the server to push inbound messages directly to the client the exact millisecond they arrive, completely bypassing the need for client-side polling.
2. **The Backend Switchboard (`active_connections`):**
   - **The Concept:** The server acts as an instantaneous router using an in-memory dictionary.
   - **Technical Detail:** Every active connection is registered in the `active_connections` dictionary, keyed directly by the user's phone number. When an outbound JSON payload arrives at the server, it doesn't need to search through active threads; it simply looks up the recipient's phone number in this dictionary and pushes the data down their specific live socket.
3. **The Asynchronous Operation (Save, then Forward):**
   - **The Concept:** We never sacrifice data integrity for speed.
   - **Technical Detail:** When a message hits the server, it executes a single asynchronous operation with two steps: first, the payload is immediately persisted to the PostgreSQL database to ensure it is never lost; second, it is forwarded to the recipient's live socket. If the recipient is offline, the server simply completes the database write and drops the forwarding step gracefully.

4. **The Reactive Frontend (Textual):**
    - **The Concept:** The terminal UI must handle incoming data without interrupting the user's typing.
    - **Technical Detail:** The Textual UI runs an asynchronous listener loop in the background. When a new payload comes through the socket, this reactive loop receives it and recalculates the screen layout. It re-renders only the conversation pane with the new message, ensuring the interface remains completely flicker-free and never blocks the user from typing.
5. **Graceful Disconnection:**
    - **The Concept:** We prevent memory leaks when users log off or lose internet.
    - **Technical Detail:** The moment a disconnection event is detected (e.g., the user closes their terminal or loses Wi-Fi), the server instantly catches the `WebSocketDisconnect` exception and purges that specific socket from the `active_connections` memory map. This prevents the server from attempting to route messages to stale, broken references.

> **How to summarize it quickly during the presentation:**
>
> "By utilizing a persistent WebSocket channel mapped in server memory, we achieve instant, push-based message delivery. When a text is sent, our backend asynchronously logs it to the database and pushes it to the recipient's socket, where the Textual frontend updates the screen instantly without ever freezing the user's terminal."

Slide 11: Testing & Diagnostics

Talking Points:

- **The Testing Methodology:** To ensure the system was robust under real-world conditions, we utilized a live, side-by-side terminal diagnostic approach. We launched the FastAPI backend in an independent terminal window, allowing us to actively monitor network traffic and security audit logs in real-time. Simultaneously, we spun up multiple client instances side-by-side to simulate concurrent users attempting to chat.
- **Authentication & Security Validations (TC-001 to TC-003):** We first rigorously tested the security handshakes.
    - We confirmed that when a client hits the `/request-otp` endpoint, the system correctly generates the token and maps it precisely into the memory cache.
    - We verified the "happy path," ensuring valid OTPs established a secure session and loaded the primary workspace.
    - Crucially, we also tested failure states: we submitted invalid and expired payload strings, verifying that the API correctly intercepted the bad data, dropped the packet, returned an HTTP 400 error, and alerted the client interface.

- **Real-Time Data Routing & Database Integrity (TC-004):** While our mock users exchanged live messages, we concurrently ran database table scans.
  - This testing proved that the system doesn't just route the JSON payload instantly to the target screen; it simultaneously creates persistent row entries within the PostgreSQL tables.
  - By verifying the disk writes, we confirmed that user profiles were perfectly linked via their IDs, ensuring that no user could ever accidentally access someone else's private chat threads.
- **System Resilience and Sync Recovery (TC-005):** Finally, we tested how the system handles network interruptions.
  - We simulated sudden terminal disconnects and forced re-logins.
  - The tests verified that upon reconnection, the client interface automatically calls the historical API endpoints, pulling the preserved data from PostgreSQL and seamlessly repopulating the chat view with the old logs. This confirms our architecture prevents any data loss, even if a socket connection unexpectedly drops.

Slide 12: Key Features of the System

Talking Points:

(Introduction) To summarize the architecture and design decisions we've discussed, here are the six core features that define this terminal-based chat application:

1. **OTP Authentication (Password-less Security)**
   - **The Feature:** We implemented a password-less login handshake using a mobile number and a one-time verification code.
   - **Technical Detail:** By completely removing static passwords, we eliminated the need to manage complex password hashing and mitigated the risk of mass credential leaks. The system verifies identity rapidly via a temporary memory cache before allowing database access.
2. **Real-Time Messaging (Zero Polling)**
   - **The Feature:** The system provides instant, one-to-one message exchange.
   - **Technical Detail:** Because we utilize persistent, full-duplex WebSocket connections, the server pushes messages the millisecond they arrive. This completely eliminates "polling"—the resource-heavy process of clients constantly asking the server if there is new data.
3. **Persistent Database History**
   - **The Feature:** Every conversation is permanently stored and can be replayed on demand between any two participants.

- **Technical Detail:** Using PostgreSQL and the SQLAlchemy ORM, we make sure that no message is ever lost. Even if a user logs in from a completely different machine, the stateless REST API pulls their historical records instantly and repopulates their screen.

4. **Dynamic Contact Management**
   - **The Feature:** The UI includes an intuitive sidebar interface for registering new target numbers and cleanly switching between active conversations.
   - **Technical Detail:** When a user switches contacts, the frontend automatically updates its state, clears the main chat pane, and issues an asynchronous fetch command to the backend to grab the correct message history.
5. **Reactive Terminal User Interface (TUI)**
   - **The Feature:** We built a CSS-styled interface using the Textual framework that re-renders asynchronously.
   - **Technical Detail:** This is not a standard scrolling command-line script. The reactive engine listens for data over the socket and repaints only the specific bounding boxes that change, creating a smooth, flicker-free experience that mimics modern web applications.
6. **Lightweight**
   - **The Feature:** The application consumes a mere fraction of the CPU and RAM resources required by bloated graphical clients like Electron-based apps.
   - **Technical Detail:** Because it operates entirely natively within the terminal shell, it allows developers, system administrators, and power users to communicate seamlessly without degrading their machine's performance or interrupting their workflow.

> **How to summarize it quickly during the presentation:**
>
> "In short, we've built a system that delivers the password-less security, persistent history, and real-time speed of a modern messaging platform, all wrapped in a reactive, incredibly lightweight terminal interface."

Slide 13: Future Scope & Conclusion

Talking Points:

To wrap up the presentation, we have successfully demonstrated that a CLI tool can overcome graphical applications in speed and features.

What's Next? Because we built this on an asynchronous ASGI foundation, scaling it is highly feasible.

- We can deploy the backend to the cloud and leverage horizontal scaling.

- We can also include read receipts for messages.
- We can implement Group Channels by altering the router to multiplex message streams to multiple connected sockets simultaneously.
- We can integrate End-to-End Encryption (using cryptographic layers like Curve25519) locally on the client before the message hits the WebSocket wire.
- Finally, we can also implement Binary File Transfers, converting files into chunked byte packets for secure terminal-to-terminal sharing.